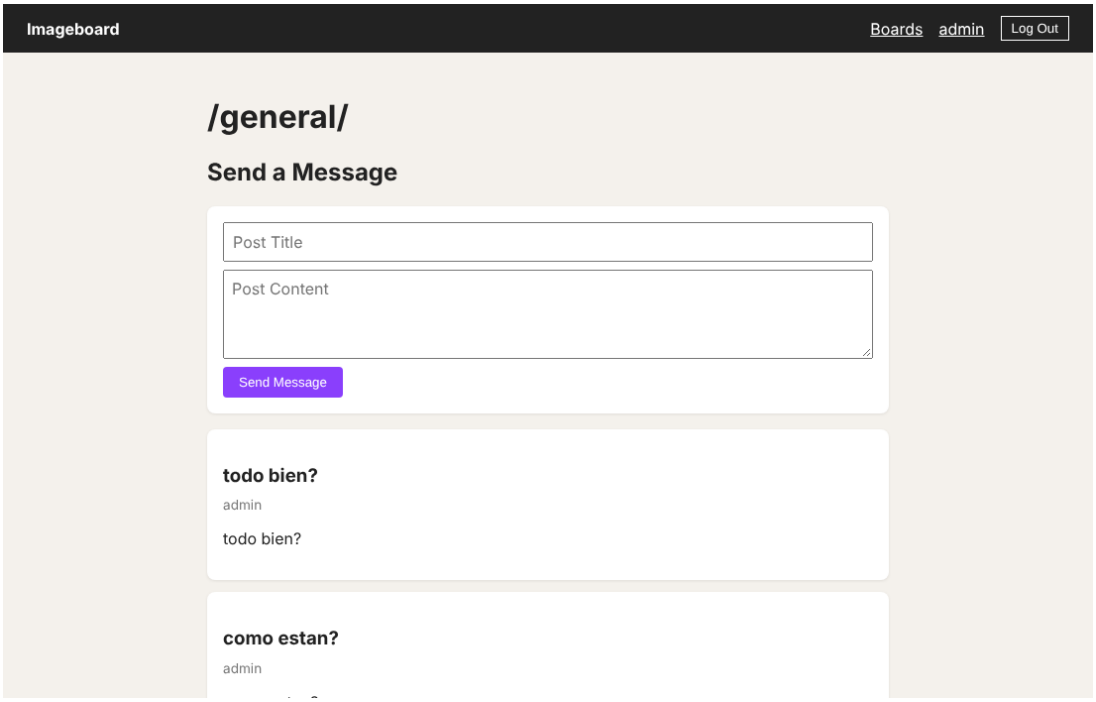


Imageboard

Tablones y publicaciones con Node.js, Express y Sequelize

Eva Gallardo Romero

23 de abril de 2025

Índice

Enlace al repositorio de GitHub **2**

1 Descripción general **2**

2 Diagrama de clases **2**

3 Diseño **2**
3.1 Lista de tablones 2
3.2 Tablón 3
3.3 Registro, login y perfil 3

4 Requisitos del sistema **3**
4.1 Requisitos funcionales 3
4.2 Requisitos no funcionales 4

5 Documentación técnica **4**
5.1 index.js 4
5.2 db/sequelize.js y db/associations.js 5
5.3 models/ 6
5.4 middleware/auth.js 6
5.5 router/users.js 7
5.6 router/auth.js 7
5.7 router/boards.js 8

Enlace al repositorio de GitHub

Puedes consultar el proyecto completo en el siguiente enlace: imageboard.

La documentación del código generada con JSDoc está en evagr11.github.io/imageboard.

1 Descripción general

Imageboard es una aplicación web al estilo de los foros por **tablones**: hay varios tablones temáticos y dentro de cada uno los usuarios publican mensajes. Cualquiera puede leer, pero para publicar o crear tablones hay que **registrarse** e **iniciar sesión**.

Es una **práctica de clase** de backend. El servidor está hecho con **Node.js** y **Express 5** y renderiza las páginas en el servidor con plantillas **Nunjucks**. Los datos se guardan en **SQLite** a través del ORM **Sequelize**, las contraseñas se guardan cifradas con **bcrypt** y las **sesiones** van en otra base de datos SQLite aparte (`connect-sqlite3`). Me sirvió para practicar rutas, modelos y relaciones, validación, autenticación, autorización y sesiones.

Al arrancar, el servidor crea las tablas desde cero y mete unos **datos de prueba**: los usuarios `admin`, `user1` y `user2`, los tablones `general`, `videojuegos` y `testing`, y tres mensajes en `general`.

2 Diagrama de clases

La aplicación sigue la estructura habitual de Express: **rutas** separadas por temas, un **middleware** que protege las rutas que necesitan sesión (marcadas con *) y tres **modelos** de Sequelize. Un usuario y un tablón tienen muchos posts, y cada post pertenece a un usuario y a un tablón.

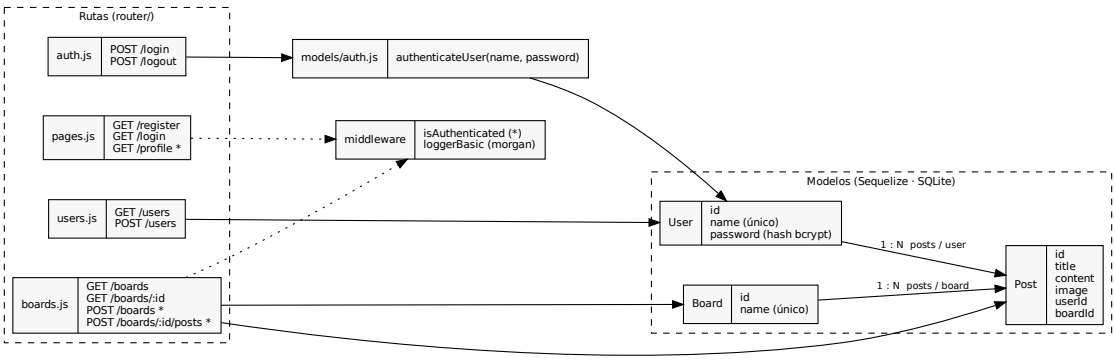


Figura 1: Rutas, modelos y relaciones del Imageboard

3 Diseño

Todas las páginas heredan de `base.njk`: una cabecera oscura con el nombre y los enlaces (*Boards*, y *Login* y *Register* o el usuario y *Log Out* si hay sesión), y el contenido centrado sobre fondo claro con tarjetas blancas. Las capturas son las plantillas del proyecto con su hoja de estilos y los datos de prueba.

3.1 Lista de tablones

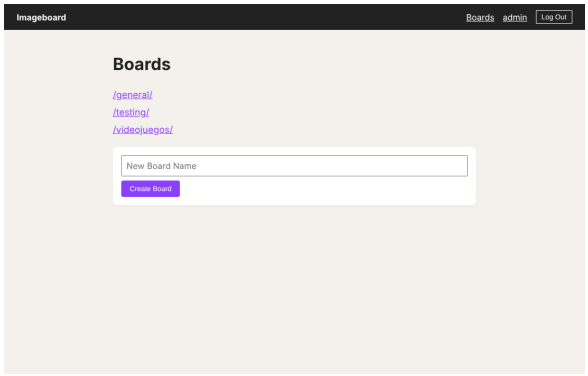
Los tablones por orden alfabético y, si hay sesión, un formulario para crear uno nuevo.

3.2 Tablón

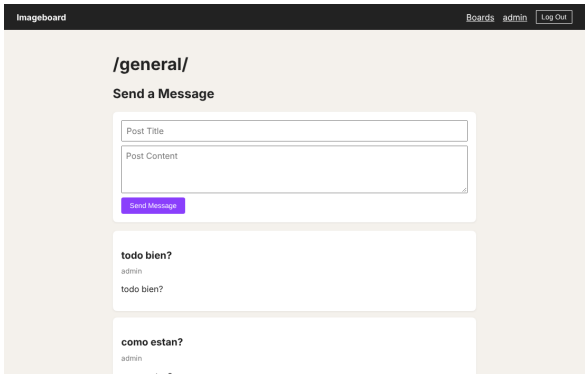
El nombre entre barras (`/general/`), el formulario para publicar si hay sesión y los mensajes del más nuevo al más antiguo con su autor.

3.3 Registro, login y perfil

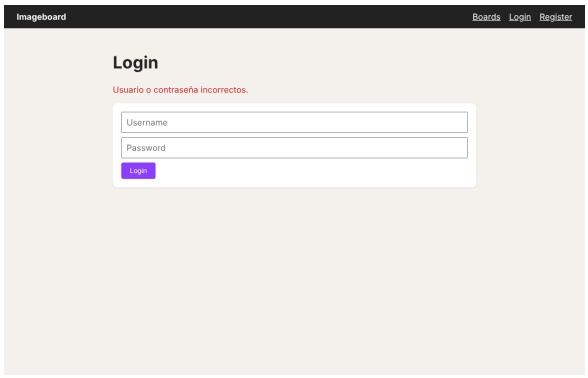
Formularios sencillos de usuario y contraseña. El login avisa si los datos no son correctos y el perfil saluda al usuario. Si se entra en una página protegida sin sesión, sale la página *Unauthorized*.



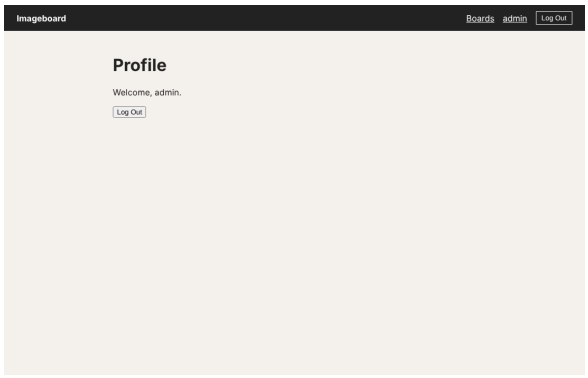
3.1. Lista de tablones



3.2. Tablón con mensajes



3.3. Login con error



3.3. Perfil

Figura 2: Páginas del Imageboard

4 Requisitos del sistema

4.1 Requisitos funcionales

ID	Requisito funcional
RF01	Cualquiera debe poder ver la **lista de tablones** y los **mensajes** de cada uno.
RF02	Un usuario se debe poder **registrar** con nombre y contraseña; el nombre no se puede repetir.
RF03	El **login** debe comprobar la contraseña con bcrypt y guardar el usuario en la sesión.
RF04	Solo los usuarios con sesión pueden **crear tablones** y **publicar**.
RF05	Un tablón con un nombre que ya existe no se debe crear dos veces.

ID	Requisito funcional
RF06	Los mensajes deben mostrarse del **más nuevo al más antiguo** con el nombre del autor.
RF07	El usuario debe poder **cerrar sesión**.
RF08	La ruta `GET /users` debe devolver los usuarios en JSON **sin la contraseña**.
RF09	Si se pide un tablón que no existe se debe responder con un **404**.

4.2 Requisitos no funcionales

ID	Requisito no funcional
RNF01	El servidor debe usar **Node.js** con **Express** y módulos ES (`import`).
RNF02	Las páginas deben renderizarse en el servidor con **Nunjucks**, con el escapado automático activado.
RNF03	Los datos deben guardarse en **SQLite** mediante el ORM **Sequelize**.
RNF04	Las contraseñas nunca se guardan en claro: se guarda su **hash de bcrypt**.
RNF05	Las sesiones deben durar un día y guardarse en una base de datos SQLite propia.
RNF06	El secreto de la sesión y el puerto se leen de **variables de entorno**.
RNF07	Cada petición debe registrarse en la consola con **morgan**.

5 Documentación técnica

Este anexo agrupa la documentación por archivos, separándola en la **ubicación** donde se encuentra, una **descripción general** y las **funciones** explicadas de manera individual con su **código** adjunto y comentado. Se omiten los `import` y los comentarios JSDoc, que repiten la descripción.

5.1 index.js

5.1.1 Ubicación

`imageboard/index.js`

Descripción general:
Punto de entrada: configura Express, las sesiones, Nunjucks y las rutas, y arranca el servidor.

5.1.2 Funciones

Configuración de la sesión **Descripción:** Las sesiones se guardan en `db/sessions.sqlite` y duran un día. El secreto se lee de una variable de entorno.

Código comentado:

```
const SQLiteStoreSession = SQLitestore(session);
const sessionStore = new SQLiteStoreSession({
  db: 'sessions.sqlite',
  dir: './db',
  table: 'sessions',
});

const sessionConfig = {
  store: sessionStore,
  secret: process.env.SESSION_SECRET || "cambia-este-secreto",
  resave: false,
  saveUninitialized: false,
  cookie: {
```

```
    maxAge: 1000 * 60 * 60 * 24, // 1 día
  }
};
```

Middlewares y rutas **Descripción:** Registra el log, la sesión, Nunjucks, los archivos estáticos y los formularios, deja el usuario disponible en todas las plantillas y monta los routers.

Código comentado:

```
app.use(loggerBasic);
app.use(session(sessionConfig));
nunjucks.configure('views', { autoescape: true, express: app });
app.use(express.static('public'));
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// el usuario de la sesión llega a todas las plantillas como currentUser
app.use((req, res, next) => {
  res.locals.currentUser = req.session.username;
  next();
});

app.set('view engine', 'njk');
app.use("/users", userRouter);
app.use("/boards", boardsRouter);
app.use("/", pagesRouter);
app.use("/", authRoutes);
app.get('/', (req, res) => res.redirect('/boards'));
```

initializeApp **Descripción:** Crea las relaciones, rehace la base de datos, mete los datos de prueba y escucha en el puerto.

Código comentado:

```
const initializeApp = async () => {
  try {
    await import('./db/associations.js');
    await sequelize.sync({ force: true });   // borra y vuelve a crear las tablas
    await import('./db/populate.js');        // datos de prueba
    app.listen(port, () => {
      console.log(`Server listening at http://localhost:${port}`);
    });
  } catch (error) {
    console.error("Error during initialization:", error);
  }
};
```

5.2 db/sequelize.js y db/associations.js

5.2.1 Ubicación

`imageboard/db`

Descripción general:
Conexión con SQLite y relaciones entre los modelos.

5.2.2 Funciones

Conexión y relaciones **Descripción:** La base de datos es el archivo `db/database.sqlite`. Un usuario y un tablón tienen muchos posts.

Código comentado:

```
export const sequelize = new Sequelize({
    dialect: "sqlite",
    storage: "db/database.sqlite",
    logging: false,                // sin mostrar el SQL en la consola
});
```

```
6
7  User.hasMany(Post, { foreignKey: 'userId', as: 'posts' });
8  Post.belongsTo(User, { foreignKey: 'userId', as: 'user' });
9  Board.hasMany(Post, { foreignKey: 'boardId', as: 'posts' });
10 Post.belongsTo(Board, { foreignKey: 'boardId', as: 'board' });
```

5.3 models/

5.3.1 Ubicación

`imageboard/models`

Descripción general:
Los modelos `User`, `Board` y `Post` definen las tablas, y `auth.js` comprueba el usuario y la contraseña.

5.3.2 Funciones

Post **Descripción:** Un post tiene título, contenido, imagen opcional y las claves del usuario y del tablón.

Código comentado:

```
1  export const Post = sequelize.define('Post', {
2      id: { type: Sequelize.INTEGER, autoIncrement: true, primaryKey: true },
3      title: { type: Sequelize.STRING, allowNull: false },
4      content: { type: Sequelize.TEXT, allowNull: false },
5      image: { type: Sequelize.STRING, allowNull: true },
6      userId: {
7          type: Sequelize.INTEGER, allowNull: false,
8          references: { model: 'users', key: 'id' }
9      },
10     boardId: {
11         type: Sequelize.INTEGER, allowNull: false,
12         references: { model: 'boards', key: 'id' }
13     },
14 }, { tableName: "posts" });
```

authenticateUser **Descripción:** Busca el usuario por nombre y compara la contraseña con su hash. Devuelve el usuario o `null`.

Código comentado:

```
1  const checkPassword = async (password, hashedPassword) => {
2      return await bcrypt.compare(password, hashedPassword);
3  }
4
5  export const authenticateUser = async (name, password) => {
6      const user = await User.findOne({ where: { name: name } });
7      if (!user) {
8          return null;                          // no existe
9      }
10     const isPasswordValid = await checkPassword(password, user.password);
11     if (!isPasswordValid) {
12         return null;                          // contraseña incorrecta
13     }
14     return user;
15 }
```

5.4 middleware/auth.js

5.4.1 Ubicación

`imageboard/middleware/auth.js`

Descripción general:
Protege las rutas que necesitan sesión.

5.4.2 Funciones

isAuthenticated **Descripción:** Deja pasar si hay usuario en la sesión; si no, responde 401 con la página *Unauthorized*.

Código comentado:

```
export const isAuthenticated = (req, res, next) => {
    if (req.session.userId) {
        return next();
    }
    res.status(401).render("unauthorized.njk", { title: "Unauthorized" });
}
```

5.5 router/users.js

5.5.1 Ubicación

`imageboard/router/users.js`

Descripción general:
Lista de usuarios en JSON y registro, que sirve tanto para el formulario como para la API.

5.5.2 Funciones

POST /users **Descripción:** Comprueba los campos, guarda la contraseña cifrada y responde según venga del formulario (redirige al login) o de la API (JSON).

Código comentado:

```
router.post('/', async (req, res) => {
    const { name, password } = req.body;
    const desdeFormulario = req.is('application/x-www-form-urlencoded');
    if (!name || !password) {
        return desdeFormulario
            ? res.status(400).render('register', { title: "Register", error: "Rellena
                usuario y contraseña." })
            : res.status(400).json({ error: "Faltan name o password." });
    }
    try {
        const hashedPassword = await bcrypt.hash(password, 10);   // 10 rondas de sal
        const user = await User.create({ name, password: hashedPassword });
        if (desdeFormulario) {
            return res.redirect('/login?registered=1');
        }
        res.status(201).json({ id: user.id, name: user.name });
    }
    catch (error) {
        const mensaje = error.name === 'SequelizeUniqueConstraintError'
            ? "Ese nombre de usuario ya existe."
            : "No se pudo crear el usuario.";
        return desdeFormulario
            ? res.status(400).render('register', { title: "Register", error: mensaje
                })
            : res.status(400).json({ error: mensaje });
    }
});
```

5.6 router/auth.js

5.6.1 Ubicación

`imageboard/router/auth.js`

Descripción general:
Inicio y cierre de sesión.

5.6.2 Funciones

POST /login Descripción: Si el usuario y la contraseña son correctos, guarda su id y nombre en la sesión y lleva al perfil.

Código comentado:

```
router.post('/login', async (req, res) => {
    try {
        const { name, password } = req.body;
        const user = await authenticateUser(name, password);
        if (!user) {
            return res.redirect('/login?error=1');
        }
        req.session.userId = user.id;
        req.session.username = user.name;
        res.redirect("/profile");
    }
    catch (error) {
        console.error(error);
        res.status(500).json({ error: "An error occurred during login." });
    }
});
```

5.7 router/boards.js

5.7.1 Ubicación

`imageboard/router/boards.js`

Descripción general:
Tablones y sus posts.

5.7.2 Funciones

GET /boards/:id Descripción: Busca el tablón y sus posts con el nombre del autor, del más nuevo al más antiguo.

Código comentado:

```
router.get('/:id', async (req, res) => {
    const board = await Board.findByPk(req.params.id);
    if (!board) {
        return res.status(404).send('Board not found');
    }
    const posts = await Post.findAll({
        where: { boardId: board.id },
        include: { model: User, as: 'user', attributes: ['name'] },   // solo el
            nombre
        order: [['createdAt', 'DESC']],
    });
    res.render('board', { title: board.name, board, posts });
});
```

POST /boards y POST /boards/:id/posts Descripción: Crear un tablón o publicar, solo con sesión. El post se asocia al usuario de la sesión.

Código comentado:

```
router.post('/', isAuthenticated, async (req, res) => {
    const name = (req.body.name || '').trim();
    if (name) {
        await Board.findOrCreate({ where: { name } });   // no duplica nombres
    }
    res.redirect('/boards');
});

router.post('/:id/posts', isAuthenticated, async (req, res) => {
    const { title, content } = req.body;
```

```
    const boardId = req.params.id;
    const board = await Board.findByPk(boardId);
    if (!board) {
        return res.status(404).send('Board not found');
    }
    await Post.create({
        title,
        content,
        boardId,
        userId: req.session.userId,
    });
    res.redirect(`/boards/${boardId}`);
});
```